

PHANTOM CONSENSUS

TEAM EXECUTION BLUEPRINT

3-Person Hackathon War Plan · Full Reference Document

P1	ARCHITECT Infrastructure · Pipeline · Data Layer · CI/CD · Config
P2	STRATEGIST Scoring · Strategy · Filters · Alliance Detection · Consensus
P3	VALIDATOR All 18 Scenario Tests · Edge Cases · Output · Dashboard

CONTAINS: Strategic Overview · 20 GitHub Issues · 30-Step Execution Plan · Algorithm Deep-Dive · Risk Analysis · Final Checklist

STRATEGIC OVERVIEW

The Phantom Consensus engine is a political agreement simulator that must navigate dirty data, hidden betrayal networks, faction infiltrators, and priority traps — all while producing a stable, defensible consensus under time pressure. This blueprint coordinates three specialists in strict parallel, eliminating merge conflicts and maximizing throughput across a 4-hour hackathon window.

Parallelism Principle

P1 pushes skeleton stubs (empty functions with correct signatures) by T+30 min so P2 and P3 can import and work independently without merge conflicts. P3 writes tests against interfaces before P2 finishes implementations — pure Test-Driven Development.

Parallel Execution Timeline

TIME	P1 — ARCHITECT	P2 — STRATEGIST	P3 — VALIDATOR
0–30 min Phase 0	Repo setup, pyproject.toml, Makefile, config.py, Pydantic models/stubs	Algorithm whiteboard, pseudocode, docs/ALGORITHM_NOTES.md	confest.py + all 18 scenario data fixtures
30–75 min Phase 1	Ingestion layer: JSON + CSV parsers, loader.py orchestrator	relationship_scorer.py + objection_scorer.py	Unit tests for sanitization (mock-driven)
75–120 min Phase 2	Sanitization: id_normalizer, type_coercion, dedup, ref_validator	viability_scorer, trojan_filter, poison_filter	Scenario tests: trojan, poison_pill, false_friend, supporter_coherence
120–165 min Phase 3	CI/CD: GitHub Actions yml, coverage gate, structlog setup	alliance_detector, infiltrator_scan, consensus_builder	Scenario tests: cascading_betrayal, infiltrator, scale, dirty_csv
165–210 min Phase 4	Profiler, Rich dashboard, --query CLI flag, APPROACH.md	Threshold calibration (data-driven), Floyd-Warshall cascading	Adversarial fuzz fixtures, Hypothesis property tests
210–240 min Phase 5	Final integration, smoke test, submission packaging	Beam search / combo enumeration for greedy fix, final review	18/18 scenario pass check, coverage gate 90%+, output verify

Ownership Boundaries

P1 — ARCHITECT	P2 — STRATEGIST	P3 — VALIDATOR
src/ingestion/	src/scoring/	tests/ (all)
src/sanitization/	src/strategy/	src/output/formatter.py
src/models/	Algorithm design	18 scenario fixtures
src/utils/	Threshold calibration	Adversarial fuzz tests
root config.py	Floyd-Warshall impl	Coverage gate (90%+)
CI/CD (.github/)	Beam search / combos	Final JSON validation
dashboard/		
APPROACH.md		

Merge Conflict Prevention Rules

P1 owns src/ingestion/ + src/sanitization/ + src/mapping/	P2 and P3 IMPORT only — never edit. File a GitHub issue, P1 fixes.
P2 owns src/scoring/ + src/strategy/	P1 and P3 never touch these files. P3 writes tests that import from them.
P3 owns tests/ + src/output/ + dashboard/	P1 and P2 never touch tests/. formatter.py is strictly P3 territory.
Shared: config.py changes	ONLY P1 edits config.py. P2 asks via chat → P1 PRs in 2 min.

GITHUB ISSUES — ALL 20

The 20 issues span five concern areas: project setup, data parsing, data sanitisation, core consensus logic, and performance. P1 owns issues 1–9 (infrastructure and data pipeline), P2 owns issues 10–17 (all scoring and strategy), and P3 owns issues 18–20 (output formatting, edge cases, and performance profiling).

#	Title	Labels	Owner
1	Project Setup and Structure	documentation, setup	P1
2	Parse Representatives Data	data-processing	P1
3	Parse Proposals Data	data-processing	P1
4	Parse Objections Data	data-processing	P1
5	Parse Relationships Data (CSV)	data-processing	P1
6	Sanitize Representative IDs	data-processing, error-handling	P1
7	Handle Invalid Attribute Types	error-handling, data-processing	P1
8	Deduplicate Proposals	data-processing, error-handling	P1
9	Validate Missing References	error-handling, data-processing	P1
10	Compute Relationship Scores	core-logic, optimization	P2
11	Calculate Objection Weights	core-logic	P2
12	Filter Trojan Horse Representatives	core-logic, error-handling	P2
13	Reject Poison Pill Proposals	core-logic	P2
14	Identify Genuine Alliances	core-logic	P2
15	Handle Asymmetric Trust	core-logic	P2
16	Uncover Faction Infiltrators	core-logic, error-handling	P2
17	Formulate Consensus Output	core-logic, optimization	P2
18	Format JSON Output	data-processing	P3
19	Handle Extreme Edge Cases	error-handling, performance	P3
20	Scale Performance Profiling	performance, optimization	P3

Issue Detail Descriptions

#1	Project Setup and Structure Initialize the repository structure with standard organizational conventions. Create directories for data ingestion, source code, and outputs. Provide a foundational scaffold allowing clear separation between data parsing, logic execution, and result formatting. No implementation logic — environment preparation only.	P1
#2	Parse Representatives Data Read and ingest representatives.json containing politician IDs, names, factions, and influence scores. Load data reliably into memory for the consensus engine. Focus only on file loading — defer sanitization to later stages.	P1

#3	Parse Proposals Data Develop logic to read proposals.json containing bill IDs, titles, sponsoring representative IDs, and priority levels. Store in a format suitable for quick lookups during evaluation. Strictly focused on raw data ingestion.	P1
#4	Parse Objections Data Implement ingestion of objections.json detailing which representatives oppose which proposals and their severity scores. Load in a way that enables quick retrieval by proposal ID or representative ID.	P1
#5	Parse Relationships Data (CSV) Read relations.csv mapping pairs of politicians with mutual trust, rivalry scores, and betrayal probabilities. Uses a distinct CSV parser approach with row-level fault isolation — bad rows are quarantined, not discarded silently.	P1
#6	Sanitize Representative IDs Normalize all representative IDs across all datasets. Handle mixed case (REP_001 vs rep_001) and extraneous whitespace. Standardize identifiers before any core logic attempts to cross-reference records.	P1
#7	Handle Invalid Attribute Types Cleanse the influence metric in representative data. Handle strings-as-integers, null values, and out-of-bounds numbers (>100). Cast types correctly, manage missing values with median imputation, and clamp or filter out-of-range inputs without crashing.	P1
#8	Deduplicate Proposals Identify and resolve duplicate entries in the proposals dataset. When duplicates exist, keep the one with the highest priority score. Ensures each unique proposal is evaluated exactly once.	P1
#9	Validate Missing References Detect "ghost" references across datasets — objections referencing non-existent representatives, proposals with missing sponsors. Validate foreign key integrity and quarantine orphaned records rather than letting them disrupt processing.	P1
#10	Compute Relationship Scores Derive actionable scores from raw trust and betrayal probability values. Formula: $\text{score} = \text{trust} \times (1 - \text{betrayal_prob}) / 100$. Build a NumPy-backed relationship matrix. Apply Floyd-Warshall transitive risk propagation to catch 3-hop betrayal chains.	P2
#11	Calculate Objection Weights Calculate a weighted metric for each objection: $\Sigma(\text{severity} \times \text{influence})$ for all objectors per proposal, normalized by dividing by max possible ($100 \times 100 = 10,000$). Representatives with null influence are excluded from the sum without crashing.	P2
#12	Filter Trojan Horse Representatives Identify representatives with high influence AND high betrayal probability. Condition: $\text{max_outgoing_betrayal} > \text{TROJAN_BETRAYAL_THRESHOLD}$ AND $\text{influence} > \text{TROJAN_INFLUENCE_THRESHOLD}$. Use data-driven thresholds calibrated to the input distribution. Excluded reps are logged but not silently dropped.	P2
#13	Reject Poison Pill Proposals Detect proposals that have high priority but face severe, widespread objections. Viability formula: $\text{priority} \times (1 - \text{controversy})$ where $\text{controversy} = \min(1.0, \text{obj_weight} / \text{controversy_threshold})$. The threshold is adaptive: $0.6 \times \text{max_obj_weight}$ in the dataset.	P2

#14	Identify Genuine Alliances Use NetworkX connected components to detect stable alliances. Add an edge only when BOTH directions of the relationship score meet the adaptive threshold. Return sorted lists for deterministic output. Uses data-driven alliance threshold: $\text{mean}(\text{scores} > 0) + 0.5 \times \text{std}$.	P2
#15	Handle Asymmetric Trust Prevent "False Friend" misclassification — where rep A highly trusts B but B distrusts A. The bidirectional score returns $\min(\text{score}_{A \rightarrow B}, \text{score}_{B \rightarrow A})$. An edge is only added to the alliance graph if both directions meet threshold.	P2
#16	Uncover Faction Infiltrators Detect spies within factions: representatives who share a faction label but have high betrayal probabilities against fellow faction members. Compute average intra-faction betrayal score. If $\text{avg_score} < (1 - \text{INFILTRATOR_THRESHOLD})$, flag and exclude from alliance candidacy.	P2
#17	Formulate Consensus Output The core decision-making loop. For ≤ 30 proposals, enumerate top-k combinations using <code>itertools.combinations</code> to find the highest-scoring set. Supporters are all safe reps who did NOT object to the selected proposals, sorted by influence descending.	P2
#18	Format JSON Output Construct the final output module. Format decisions into the required JSON structure: selected proposals, supporting representatives, and detected alliances. Validate output against Pydantic schema before writing. Save to <code>output/final_agreement.json</code> .	P3
#19	Handle Extreme Edge Cases Ensure stability under: complete rivalry (all scores below threshold $\rightarrow$ alliances: <code>[]</code>), single representative + single proposal (minimum viable case), circular trust chains (no infinite loops), and all reps in one faction.	P3
#20	Scale Performance Profiling Profile the consensus engine at $n=50$ representatives, 30 proposals. Score matrix build must be $< 10\text{ms}$ (NumPy vectorized). Full pipeline must be $< 500\text{ms}$. Use <code>--profile</code> CLI flag wrapping <code>cProfile</code> . Optimize nested loops and redundant calculations.	P3

30-STEP EXECUTION PLAN

PHASE 0 · T+0 → T+30 MIN

IMMEDIATE PARALLEL BLAST

STEP 1	P1: Git Repo + Project Skeleton	P1
• Create GitHub repo, push initial commit with .gitignore, README stub • Create pyproject.toml with all dependencies: numpy, pandas, networkx, pydantic, structlog, rich, typer, pytest • Create Makefile with targets: run, test, lint, format, dashboard, clean • Create src/__init__.py and all subdirectory __init__.py files • Create src/config.py with ALL constants and docstring rationale for each threshold <pre>python -c 'from src.config import TROJAN_BETRAYAL_THRESHOLD; print(TROJAN_BETRAYAL_THRESHOLD)' # must not crash</pre>		
STEP 2	P1: Pydantic Models — Shared Contract	P1
• Create src/models/representative.py — Representative + ValidatedRep with influence coercion • Create src/models/proposal.py — Proposal with priority field • Create src/models/objection.py — Objection with severity + rep_id • Create src/models/relationship.py — Relationship + RelationshipScore dataclass • Push to main immediately — this unlocks P2 and P3 <pre>python -c "from src.models.representative import Representative; r=Representative(id='rep_001',name='x',faction='A',influence='85'); assert r.influence=='85'"</pre>		
STEP 3	P2: Algorithm Whiteboard + Pseudocode	P2
• DO NOT write code yet — spend first 30 min on algorithm design • Write pseudocode for relationship_scorer: $\text{trust} \times (1 - \text{betrayal_prob}) / 100$ • Write pseudocode for alliance_detector: bidirectional min-score graph + connected components • Map all 18 hidden test scenarios to the specific module that catches each one • Identify cascading betrayal fix: Floyd-Warshall or damped BFS depth 3 • Identify threshold calibration fix: $\text{mean} + 0.5 \times \text{std}$ from input data • Write in docs/ALGORITHM_NOTES.md and push to repo		
STEP 4	P3: conftest.py + All 18 Scenario Fixtures	P3
• Create tests/conftest.py with build_scenario() factory and run_engine() fixture • Create tests/scenarios/ directory with one file per hidden test scenario • Each scenario has crafted JSON data that WILL trigger that specific trap • Tests WILL FAIL until P2 finishes — that is expected and correct (TDD) • This is the most important P3 task — spend 25 minutes here <pre>pytest tests/scenarios/ --collect-only # must collect 18 files without import errors</pre>		

PHASE 1 · T+30 → T+75 MIN

DATA INGESTION LAYER

STEP 5 P1: JSON Parser (reps, proposals, objections)**P1**

- Create src/ingestion/json_parser.py with load_representatives, load_proposals, load_objections
- Each function: open file, json.load, wrap in try/except, return empty list on failure
- Log file load success/failure with structlog (event='file_load_failed')
- Log success with event='file_loaded' and record_count

```
pytest tests/unit/test_json_parser.py # test valid + invalid + missing files
```

STEP 6 P1: CSV Parser (relations.csv) — Row-Level Fault Isolation**P1**

- Create src/ingestion/csv_parser.py with load_relationships(path) → list[dict]
- BAD ROWS go to quarantine log — they do NOT crash the parser
- Use pandas read_csv with dtype=str to avoid silent type coercion
- Log quarantined rows with row number and error
- This is the "Dirty CSV Parsing" hidden test — row-level isolation is the key

```
pytest tests/unit/test_csv_parser.py -v # include fixture CSV with 1 bad row
```

STEP 7 P1: Loader Orchestrator**P1**

- Create src/ingestion/loader.py — single entry point calling all parsers
- load_all(data_dir: Path) → RawData TypedDict with all 4 lists
- Push immediately — P2 can now write scoring code that uses RawData

```
python -c "from src.ingestion.loader import load_all; from pathlib import Path; d=load_all(Path('data')); print(d.keys())"
```

STEP 8 P2: Relationship Scorer (starts T+30 after models push)**P2**

- Create src/scoring/relationship_scorer.py
- Formula: $\text{score}[i][j] = \text{trust} \times (1 - \text{betrayal_prob}) / 100$, clamped to [0,1]
- build_score_matrix(relationships, rep_ids) → RelationshipMatrix (NumPy backed)
- get_bidirectional_score(matrix, a, b) → $\min(\text{score}[a \rightarrow b], \text{score}[b \rightarrow a])$
- Handle missing pairs: default to 0.0 score

```
pytest tests/unit/test_relationship_scorer.py -v
```

STEP 9 P2: Objection Weight Calculator**P2**

- Create src/scoring/objection_scorer.py
- Formula: $\Sigma(\text{severity}_i \times \text{influence}_i)$ for all objectors per proposal
- Normalize to [0,1] by dividing by 10,000 (100 severity × 100 influence)
- If influence is None: skip that objector — do NOT crash
- Return 0.0 for proposals with no objections

```
pytest tests/unit/test_objection_scorer.py -v
```

STEP 10 P3: Unit Tests for Sanitization (mock-driven)**P3**

- Create tests/unit/test_id_normalizer.py — test all ID format variations
- Create tests/unit/test_type_coercion.py — influence casting edge cases
- Create tests/unit/test_deduplicator.py — duplicate detection and keep-rule
- Create tests/unit/test_ref_validator.py — ghost reference detection
- Use @pytest.mark.parametrize for bulk test cases
- These tests run independently of P2 — zero dependency

```
pytest tests/unit/ --tb=short # expect import errors until P1 pushes, not test logic errors
```

PHASE 2 · T+75 → T+120 MIN**SANITIZATION LAYER**

STEP 11 **P1: ID Normalizer (Issue 6)****P1**

- Create `src/sanitization/id_normalizer.py`
- `normalize_rep_id(raw)` → canonical 'rep_001' format (strip, lower, zfill to 3 digits)
- Handle: REP_001, rep_1, Rep-002, " REP_1 ", REP001 → all → rep_001 format
- `normalize_prop_id(raw)` → lowercase with underscores
- Run normalizer on ALL datasets before any cross-referencing

```
pytest tests/unit/test_id_normalizer.py -v # should now all pass
```

STEP 12 **P1: Type Coercion + Null Handler (Issue 7)****P1**

- Create `src/sanitization/type_coercion.py`
- `coerce_influence(v)` → `int | None`: try `int(float(str(v).strip()))`, except return `None`
- Clamp result: `max(0, min(100, parsed))`
- Implement median imputation for `None` influence (collect valid values, compute median)
- Records that fail go to quarantine log — not dropped silently

```
pytest tests/unit/test_type_coercion.py -v # all edge cases must pass
```

STEP 13 **P1: Deduplicator + FK Validator (Issues 8 & 9)****P1**

- Create `src/sanitization/deduplicator.py` — deduplicate proposals by ID, keep highest priority
- Create `src/sanitization/ref_validator.py` — validate all FK references across datasets
- Ghost sponsors: proposal references `rep_id` not in representatives → quarantine proposal
- Ghost objectors: objection references `rep_id` not in representatives → quarantine objection
- All quarantined records emitted to structured quarantine log

```
pytest tests/unit/test_deduplicator.py tests/unit/test_ref_validator.py -v
```

STEP 14 **P2: Viability Scorer (Issue 13)****P2**

- Create `src/scoring/viability_scorer.py`
- Formula: $\text{viability} = (\text{priority} / 10) \times (1 - \text{controversy})$
- $\text{controversy} = \min(1.0, \text{obj_weight} / \text{controversy_threshold})$
- $\text{controversy_threshold} = 0.6 \times \max(\text{obj_weights.values()})$ — adaptive to dataset
- This is the adaptive Poison Pill detection fix

```
pytest tests/unit/test_viability_scorer.py -v
```

STEP 15 **P3: Scenario Tests — Core 4****P3**

- Complete `tests/scenarios/test_trojan_horse.py` — 2 assertions
- Complete `tests/scenarios/test_poison_pill.py` — high priority + heavy objection → rejected
- Complete `tests/scenarios/test_false_friend.py` — asymmetric trust → no alliance
- Complete `tests/scenarios/test_supporter_coherence.py` — objector never in supporting_reps
- These 4 tests cover ~24 points of the hidden test suite

```
pytest tests/scenarios/test_trojan_horse.py tests/scenarios/test_poison_pill.py -v
```

PHASE 3 · T+120 → T+165 MIN

STRATEGY LAYER

STEP 16 P2: Trojan Filter (Issue 12)**P2**

- Create `src/strategy/trojan_filter.py`
- Condition: `max_outgoing_betrayal > TROJAN_BETRAYAL_THRESHOLD AND influence > TROJAN_INFLUENCE_THRESHOLD`
- Use data-driven thresholds from `threshold_calibrator` (not hardcoded)
- `filter_trojans(reps, matrix) → list[Representative]` (safe reps only)
- Log each exclusion: `event='trojan_detected', rep_id, influence, max_betrayal`

```
pytest tests/scenarios/test_trojan_horse.py -v # must pass now
```

STEP 17 P2: Alliance Detector with Asymmetric Trust Fix (Issues 14 & 15)**P2**

- Create `src/strategy/alliance_detector.py` using `NetworkX`
- Build Graph — edge exists ONLY if BOTH directions $\geq$ `ALLIANCE_MIN_SCORE`
- Data-driven threshold: `ALLIANCE_MIN_SCORE = mean(all_scores > 0) + 0.5 × std`
- Connected components with ≥ 2 members = genuine alliances
- Return sorted lists for deterministic output

```
pytest tests/scenarios/test_false_friend.py -v # must pass
```

STEP 18 P2: Faction Infiltrator Scanner (Issue 16)**P2**

- Create `src/strategy/infiltrator_scan.py`
- Group reps by faction; for each member compute avg score against all other faction members
- If `avg_score < (1 - INFILTRATOR_THRESHOLD)` → flagged as infiltrator
- Flagged reps excluded from alliance candidacy within that faction
- Log `event='infiltrator_detected'`

```
pytest tests/scenarios/test_faction_infiltrator.py -v
```

STEP 19 P2: Cascading Betrayal Fix — Floyd-Warshall (CRITICAL)**P2**

- CRITICAL: Original BFS depth-2 misses 3-hop betrayal chains
- Implement transitive risk: `risk[i][j] = 1 - score[i][j]`
- Floyd-Warshall: for `k,i,j`: `risk[i][j] = min(risk[i][j], risk[i][k] + risk[k][j])`
- Convert back: `transitive_score[i][j] = 1 - risk[i][j]`
- Use transitive matrix in `trojan_filter` AND `alliance_detector`

```
pytest tests/scenarios/test_cascading_betrayal.py -v # 3-hop chain must be caught
```

STEP 20 P1: CI/CD GitHub Actions + Structlog Config**P1**

- Create `.github/workflows/ci.yml` — `ruff lint, mypy --strict, pytest 90% coverage gate`
- Create `src/utils/logger.py` — structlog config in JSON mode (`LOG_FORMAT env var`)
- Create `src/utils/profiler.py` — `cProfile` wrapper logging per-stage timing
- Push CI — it will show red until all tests pass, that is expected

```
python -c 'from src.utils.logger import get_logger; log=get_logger(); log.info("test", event="ci_check")'
```

STEP 21 P3: Scenario Tests — Extended 5**P3**

- Complete `test_cascading_betrayal.py` — 3-hop betrayal chain
- Complete `test_faction_infiltrator.py` — intra-faction spy
- Complete `test_scale_correctness.py` — 50 reps, 30 proposals, verify correct output
- Complete `test_dirty_csv.py` — CSV with 1 malformed row, rest processed
- Complete `test_id_normalization.py` — mixed case IDs cross-file

```
pytest tests/scenarios/test_cascading_betrayal.py tests/scenarios/test_dirty_csv.py -v
```

STEP 22 P2: Consensus Builder + Greedy Fix (Issue 17)

P2

- Create `src/strategy/consensus_builder.py`
- For $p \leq 30$ proposals: enumerate top-k combinations using `itertools.combinations`
- Score each combination: sum of viability scores
- Select combo with largest, most faction-diverse supporter set
- ENFORCE: objectors to selected proposals are EXCLUDED from `supporting_reps`

```
pytest tests/scenarios/test_alliance_hack.py tests/scenarios/test_minimum_viable.py -v
```

STEP 23 P2: Data-Driven Threshold Calibration (HIGH RISK FIX)

P2

- Create `src/scoring/threshold_calibrator.py`
- TROJAN threshold = $\text{mean}(\text{all betrayal_probs}) + 1.0 \times \text{std}(\text{betrayal_probs})$, clamped $[0.6-0.95]$
- ALLIANCE threshold = $\text{mean}(\text{all scores} > 0) + 0.5 \times \text{std}(\text{scores} > 0)$, clamped $[0.3-0.8]$
- VIABILITY threshold = $\text{max}(0.05, \text{mean}(\text{viability_scores}) - 0.5 \times \text{std})$
- These REPLACE hardcoded constants at runtime — `config.py` values become fallbacks
- This is the difference between 16/18 and 18/18 on hidden tests

```
python -c 'from src.scoring.threshold_calibrator import calibrate_thresholds; print("importable")'
```

STEP 24 P1: Main Entry Point + --query CLI Flag

P1

- Create `consensus_engine.py` — CLI entry point using Typer
- Main command: run full 14-stage pipeline, write `output/final_agreement.json`
- Flags: `--verbose` (ConsoleRenderer), `--profile` (cProfile output), `--dry-run`
- `--query rep_007`: print full decision trace (trojan? alliance? excluded because?)

```
python consensus_engine.py && cat output/final_agreement.json | python -m json.tool
```

STEP 25 P1: Rich Terminal Dashboard (Issue 20 polish)

P1

- Create `dashboard/visualize.py` using Rich
- Panel 1: Data Pipeline with progress table and stage record counts
- Panel 2: Threats Detected — trojans and poison pills
- Panel 3: Alliances — each alliance with member count
- Panel 4: Final Agreement — selected proposals and supporting rep count
- `--html` flag exports dashboard as standalone HTML for submission screenshots

STEP 26 P3: Adversarial Fuzz Fixtures + Hypothesis Tests

P3

- Circular trust chains: $A \rightarrow B \rightarrow C \rightarrow A$ at 0.9 — no infinite loop
- All reps in one faction — infiltrator scan handles single-faction correctly
- All reps are rivals (all scores $<$ threshold) — output: `alliances: []`
- Proposals sponsored by each other's objectors — FK validation handles this
- Single representative, single proposal — minimum viable case
- Hypothesis tests: output `rep_ids` always subset of input `rep_ids`, no duplicates in output

```
pytest tests/scenarios/test_adversarial.py -v # all 5 must pass
```

STEP 27 P3: Full 18-Scenario Run + Coverage Gate**P3**

- Run ALL 18 scenario tests: must be 18 PASSED, 0 FAILED
- Run coverage gate: `pytest tests/ --cov=src --cov-fail-under=90`
- If coverage below 90%: identify uncovered modules, add targeted unit tests
- Run full pipeline on actual sample data: `python consensus_engine.py`
- Validate output/final_agreement.json against Pydantic schema

```
pytest tests/scenarios/ -v --tb=short
```

```
python -c "from src.output.formatter import ConsensusOutput; import json; ConsensusOutput(**json.load(open('output/final_agreement.json'))); print('SCHEMA OK')"
```

STEP 28 P2: Final Algorithm Review + Performance Check**P2**

- Run `python consensus_engine.py --profile` — check per-stage timing
- Score matrix build must be <10ms at n=50 (NumPy vectorized)
- Full pipeline must be <500ms at n=50
- Verify: thresholds are calibrated (data-driven), not hardcoded
- Verify: Floyd-Warshall is called before `trojan_filter`
- Verify: combination enumeration is used for proposals ≤ 30

```
grep -r 'compute_transitive_risk' src/ # must appear in pipeline
```

STEP 29 P1: Docs + Submission Packaging**P1**

- Write APPROACH.md: 14-stage pipeline, key formulas, design decisions
- Write LIMITATIONS.md: honest self-critique
- Write RESULTS.md: what the engine outputs on sample data
- Verify README.md has install instructions, run commands, test commands
- Tag final commit: `git tag v1.0.0-hackathon && git push --tags`
- Zip repo for submission, verify zip extracts cleanly

STEP 30 ALL: Final Pre-Submission Checklist**ALL**

- `python consensus_engine.py` exits 0 and produces valid JSON
- `pytest tests/scenarios/ -v` shows 18/18 PASSED
- `pytest tests/ --cov-fail-under=90` exits 0
- `ruff check src/ tests/` exits 0 (no lint errors)
- `mypy src/ --strict` exits 0 (no type errors)
- Threshold calibration is DATA-DRIVEN (not hardcoded 0.7)
- Floyd-Warshall transitive risk is in the pipeline
- Combination enumeration replaces greedy for $p \leq 30$
- Quarantine log shows in structured logs
- `--query` CLI flag works: `python consensus_engine.py --query rep_001`
- Dashboard: `python dashboard/visualize.py` runs without error
- CI/CD: GitHub Actions workflow is committed

ALGORITHM DEEP-DIVE

1. Relationship Scoring Formula

The fundamental insight is that a high trust score is meaningless if betrayal probability is also high. The pairwise score collapses both signals into a single metric:

```
score[i][j] = trust[i][j] × (1.0 - betrayal_prob[i][j]) / 100.0
```

This formula is bounded to [0, 1]. A representative who is fully trusted (trust=100) but has a 100% betrayal probability yields a score of 0.0. A fully trusted, fully loyal representative yields 1.0.

2. Floyd-Warshall Transitive Risk Propagation

The naive approach only catches direct betrayal (depth 1). The hidden "cascading betrayal" test uses a 3-hop chain: rep_A trusts rep_B (score=0.9), rep_B trusts rep_C (score=0.9), rep_C has betrayal_prob=0.98. Depth-2 BFS misses rep_A's indirect risk through B→C. Floyd-Warshall propagates risk transitively:

```
risk[i][j] = 1.0 - score[i][j]
for k in range(n):
    for i in range(n):
        for j in range(n):
            risk[i][j] = min(risk[i][j], risk[i][k] + risk[k][j])
transitive_score[i][j] = max(0.0, 1.0 - risk[i][j])
```

Run on NumPy arrays (vectorized) for O(n³) performance with constant factor ~1ns/op. At n=50, this is 125,000 operations — <1ms in practice.

3. Data-Driven Threshold Calibration

Hardcoded thresholds (e.g. 0.7) will misclassify on distributions not centered around 0.7. All thresholds are computed at runtime from the actual input data:

Threshold	Formula	Clamp
TROJAN_BETRAYAL	mean(betrayal_probs) + 1.0 × std(betrayal_probs)	[0.60, 0.95]
ALLIANCE_MIN_SCORE	mean(scores > 0) + 0.5 × std(scores > 0)	[0.30, 0.80]
VIABILITY_MIN	max(0.05, mean(viability) - 0.5 × std(viability))	[0.05, ∞]
TROJAN_INFLUENCE	config default (75)	fixed
INFILTRATOR	config default (0.60)	fixed

4. Alliance Detection — Bidirectional Requirement

The "False Friend" trap: rep_A trusts rep_B at score=0.85, but rep_B only trusts rep_A at score=0.12. Naive graph construction (threshold on either direction) would create an edge. The fix: bidirectional score = min(score[A→B], score[B→A]). An alliance edge is only added when BOTH directions independently meet the adaptive threshold. NetworkX connected components then identify stable blocs of ≥2 members.

5. Viability Scoring — Adaptive Poison Pill Detection

A proposal with $\text{priority}=10/10$ but objections from three reps with $\text{influence}=90$ each and $\text{severity}=10$ each generates an objection weight of $3 \times 90 \times 10 / 10,000 = 0.27$. Whether this is "poison pill" depends on the dataset distribution. The adaptive controversy threshold $= 0.6 \times \max(\text{obj_weights})$ adapts to the current dataset.

```
controversy = min(1.0, obj_weight / (0.6 * max_obj_weight))
```

```
viability = (priority / 10.0) * (1.0 - controversy)
```

6. Consensus Builder — Combination Enumeration

Greedy selection (always pick the highest-viability proposal) is vulnerable to the "Alliance Hack" where the best individual proposal destroys the supporter pool. For $p \leq 30$ proposals, enumerate all combinations of up to MAX_PROPOSALS (typically 3): $C(30, 3) = 4,060$ combinations. Each is scored by sum of viability scores. The winning combination maximizes viability while preserving the largest, most faction-diverse supporter set.

RISK ANALYSIS & MITIGATIONS

Five residual risks remain after the plan's critical fixes are applied. Each has a defined mitigation strategy. Risks are rated HIGH (blocks submission), MED (costs hidden test points), or LOW (cosmetic).

#	Risk	Severity	Mitigation
1	Floyd-Warshall on 500 reps (O(n³) = 125M ops in Python)	MED	Use NumPy vectorized outer loop, not triply-nested Python. Add config flag FW
2	Combination enumeration limit unclear (MAX_PROPOSALS not specified)	MED	Add MAX_PROPOSALS = 3 to config.py. For p=30: C(30,3)=4,060 — fine. If h
3	P2 overload — 10+ complex modules critical path dependency	HIGH	P3 writes scenario tests first (Step 4) giving P2 clear targets. P2 can stub all fu
4	All influences are null — median imputation breaks (no valid values)	LOW	In coerce_all_representatives: if len(valid_influences) == 0, fall back to NULL_I
5	Time allocation — 30 steps in 4 hours (ambitious pacing)	HIGH	Steps 1-7 (infrastructure) are the true critical path. Everything else flows from t

Hidden Test Scenario Map

Each of the 18 hidden test scenarios maps to a specific module. If that module is correct, the test passes.

Scenario	Primary Module	Key Formula/Logic	Owner
Trojan Horse	trojan_filter.py	influence > THRESH AND betrayal > THRESH	P2
Poison Pill	viability_scorer.py	viability = priority × (1 – controversy)	P2
False Friend	alliance_detector.py	bidirectional min-score edge requirement	P2
Cascading Betrayal	relationship_scorer.py	Floyd-Warshall transitive risk	P2
Faction Infiltrator	infiltrator_scan.py	avg intra-faction betrayal score	P2
Dirty CSV	csv_parser.py	row-level try/except, quarantine log	P1
ID Normalization	id_normalizer.py	regex + zfill(3) canonical format	P1
Null Influence	type_coercion.py	median imputation on valid values	P1
Ghost Sponsor	ref_validator.py	FK validation → quarantine proposal	P1
Duplicate Proposal	deduplicator.py	keep highest priority on dedup	P1
Supporter Coherence	consensus_builder.py	exclude objectors from supporter set	P2
Alliance Hack	consensus_builder.py	combination enumeration vs greedy	P2
Scale Correctness	relationship_scorer.py	NumPy matrix at n=50	P2
Complete Rivalry	alliance_detector.py	all scores below threshold → alliances: []	P2
Minimum Viable	consensus_builder.py	1 rep, 1 proposal edge case	P2
Circular Trust	relationship_scorer.py	Floyd-Warshall terminates (no inf loop)	P2
Output Schema	formatter.py	Pydantic ConsensusOutput validation	P3
All-Null Influence	type_coercion.py	fallback to config default, no crash	P1

FINAL PRE-SUBMISSION CHECKLIST

Every item below must be verified before submission. Run "make all" to execute lint + test + run in a single command. A single failing item can drop score by 5-10 points.

■	python consensus_engine.py exits 0 and produces valid output/final_agreement.json
■	pytest tests/scenarios/ -v shows exactly 18/18 PASSED, 0 FAILED
■	pytest tests/ --cov=src --cov-fail-under=90 exits 0
■	ruff check src/ tests/ exits 0 — zero lint errors
■	mypy src/ --strict exits 0 — zero type errors
■	output/final_agreement.json passes Pydantic ConsensusOutput schema validation
■	APPROACH.md, LIMITATIONS.md, RESULTS.md exist and are non-trivial (>200 words each)
■	Threshold calibration is DATA-DRIVEN — grep for hardcoded 0.7 returns nothing
■	Floyd-Warshall compute_transitive_risk appears in the main pipeline
■	Combination enumeration (itertools.combinations) replaces greedy for $p \leq 30$
■	Quarantine log entries appear in structured logs — demonstrates dirty data handling
■	--query CLI flag works: python consensus_engine.py --query rep_001 prints decision trace
■	python dashboard/visualize.py runs without error and displays final agreement
■	CI/CD: GitHub Actions workflow .github/workflows/ci.yml is committed
■	git tag v1.0.0-hackathon pushed — final submission version is tagged
■	Zip archive extracts cleanly, make run produces output in fresh environment

EXECUTE THIS PLAN COMPLETELY → 18/18 HIDDEN TESTS → FIRST PLACE